

1

```
//4. (3p) Pentru functia de mai jos, sa se figureze cadrul functiei,
// iar apoi sa se genereze codul corespunzator pentru o masina virtuala bazata pe stiva.

int h(float x, int n){
    if(n<x){
        int i=0.5*(1+h(-n, x/(n/2)));
        while(i){
            x=x/(3.14-n*h(x-i, n*5));
            i=i-x;
        }
    }else return h(-1,x);
    return x;
}
```

STIVĂ

- MV Stivă, cadrul fd.

Solution:

// Cadrul functiei

x	n	int_addr	old_Fp	i
-3	-2	-1	0	1 // Fr

ENTER 1 // var loc i

FPCOMP.i -2 // n

PUSH.i -1

MUL.i

CONV.i.f.

// if (n < x)

FPCOMP.i -2 // n

CONV.i.f

FPCOMP.i -3 // x

LESS.f

IF L1 // ptr else

PUSH.f 0.5

PUSH.i 1

FPCOMP.f -3 // x

FPCOMP.i -2 // n

PUSH.i 2

DIV.i

CONV.i.f

DIV.f

CONV.f.i

CALL h
 AND.i
 CONV.i.f
 MUL.f
 CONV.f.i
 FPSTORE.i 1

// while(i)

L2:

FPLDAR.i 1 //i
 IF L3

FPLDAR.f -3 //X
 PUSH.f 3.14
 FPLDAR.i -2 //m

FPLDAR.f -3 //X
 FPLDAR.i 1 //i

CONV.i.f
 SUB.f //X-i
 FPLDAR.i -2 //m
 PUSH.i 5
 MUL.i

CALL h
 MUL.i
 CONV.i.f
 SUB.f
 DIV.f
 FPSTORE.f -3

// i = i - X

FPLDAR.i 1 //i

CONV.i.f

FPLDAR.f -3 //X

SUB.f
 CONV.f.i

FPSTORE.i 1
 JMP L1

// else

L1:

PUSH.i -1
 CONV.i.f
 FPLDAR.f -3 //X
 CONV.f.i
 CALL h
 RET.i

dec i on
 int = int - float
 for conv int - float

// return x
L3:

FPLCMN.f -3 //x

CONV.f.i

RET.i

②

```
int h(float x, float y, float z){
    if(y<x) z=-0.25+h(y*(3-x),0,z+x/2);
    else {
        int i=x;
        while(i){
            z=y*(z-2.5)/3;
            i=i*0.75;
        }
    }
    return z/5;
}
```

- MV can divide
- control flow

// control flow.

x	y	z	ret. addr	old - fp	i	
-4	-3	-2	-1	0	1	// fp

ENTER 1

// if (y < x)

FPLCMN.f -3 //y

FPLCMN.f -4 //x

LESS.f

IF L1

PUSH.f -0.25

FPLCMN.f -3 //y

PUSH.i 3

CONV.x.f

FPLCMN.f -4 //x

SUB.f

MUL.f

PUSH.i 0

CONV.x.f

FPCOMP.f -2 //z
FPCOMP.f -4 //x
PUSH.i 2

CONV.i.f

DIV.f

ADD.f

CALL L

ADD.i

CONV.i.f

FPSTORE.f -2
jmp L3

// else

L1:

FPCOMP.i 1 //i

FPCOMP.f -4 //x

CONV.f.i

FPSTORE.i 1 //i=x

// while

L2:

FPCOMP.i 1 //i

if L3

FPCOMP.f -3 //y

FPCOMP.f -2 //z

PUSH.f 2.5

SUB.f

MUL.f

PUSH.i 3

CONV.i.f

DIV.f

FPCOMP.i 1 //i

CONV.i.f

PUSH.f 0.75

MUL.f

CONV.f.i

FPSTORE.i 1 //i=...

JMP L2

return

L3:

FPCOMP.f -2 //z

PUSH.i 5

CONV.i.f

DIV.f

CONV.f.i

RET.i

3

```
// int f(int n, float x, float y){
float f(int n, float x, float y){
    int k=x*(y+2*(f(-x,0,y/(n+0.5))-1));
    while (k<n){
        k=x*(k-y);
        if(k<0 x=-f(n,y,k)/2);
        else if(k>0) x=x-1;
        else return 0;
    }
    return k;
}
```

-mv on xva
-adnl fd.

//adnl fd.

n	x	y	ret-addr	add-fr	k	
-4	-3	-2	-1	0	1	//Fr

ENTER 1

FLOAD.f -3 //x
FLOAD.f -2 //y
PUSH.i 2
CONV.i.f

FLOAD.f -3 //x
PUSH.i -1
CONV.i.f
MUL.f
CONV.f.i

PUSH.i 0
CONV.i.f

FLOAD.f -2 //y
FLOAD.i -4 //m
CONV.i.f
PUSH.f 0.5
ADD.f
DIV.f
CALL.f
PUSH.i 1
CONV.i.f
SUB.f
MUL.f
ADD.f
MUL.f
CONV.f.i
FSTORE.i 1 //k=...

// whole

L1:

FPCOMP.i 1 //R

FPCOMP.i -4 //M

LESS.i

IF L2

FPCOMP.f -3 //X

FPCOMP.i 1 //R

CONV.i.f

FPCOMP.f -2 //Y

SUB.f

MUL.f

CONV.f.i

FSTORE.i 1 //R = ...

// if

FPCOMP.i 1 //R

PUSH.i 0

LESS.i

IF L3 // do else if

FPCOMP.i -4 //M

FPCOMP.f -2 //Y

FPCOMP.i 1 //R

CONV.f.f

PUSH.i 2

CONV.i.f

DIV.f

PUSH.f -1

MUL.f

FSTORE.f -3 //X = ...
// if
L3:

FPCOMP.i 1 //R

PUSH.i 0

GREATER.i

IF L4 // alt else

FPCOMP.f -3 //X

PUSH.i 1

CONV.f.f

SUB.f

FSTORE.f -3 //X = ...
// if
L5:

PUSH.i 0

CONV.i.f

RET.f

L2:

FPCOMP.i 1

CONV.i.f

RET.f

OPTR STIVĂ

LESS.

GREATER.

EQUAL.

NOTEQ.

LESS EQ.

GREATER EQ.

REGISTRI

①

//4. (3p) Pentru funcția de mai jos, să se figureze cadrul funcției cu codul lor și dați un exemplu.
// iar apoi să se genereze codul corespunzător pentru o mașină virtuală bazată pe stivă.

```
int h(float x, int n){  
    if(n < x){  
        int i = 0.5 * (1 + h(-n, x / (n/2)));  
        while(i){  
            x = x / (3.14 - n * h(x - i, n * 5));  
            i = i - x;  
        }  
    } else return h(-1, x);  
    return x;  
}
```

// codul fd.

call_dst	x	n	ret_addr	old_FP	i	R2	R3
# -4	# -3	# -2	# -1	# 0	# 1	# 2	# 3

ENTER 1+2

// of (n < x)

CONV.i.f #2, #2
LESS.f #2, #2, #3

JF #2, L1 // else

MUL.i #2, #2, -1
CONV.i.f #2, #2

DIV.i #3, #2, 2
CONV.i.f #3, #3
DIV.f #3, #3, #3
CONV.f.i #3, #3

CALL.i #2, h, #2, #3

ADD.i #2, 1, #2

CONV.i.f #2, #2

MUL.f #2, 0.5, #2

CONV.f.i #1, #2

// while

L2:

JF #1, L3

CONV.i.f #2, #1

SUB.f #2, #3, #2

MUL.i #3, #2, 5

CALL.i #2, h, #2, #3

MUL.i #2, #2, #2

CONV.i.f #2, #2

SUB.f #2, 3.14, #2

DIV.f #3, #3, #2

CONV.i.f #2, #1

SUB.f #2, #2, #3

CONV.f.i #1, #2

JMP L2

// else

L1:

CONV.i.f #2, -1

CONV.f.i #3, #3

CALL.i #2, h, #2, #3

RET.i #2

L3:

CONV.f.i #2, #3

RET.i #2

①

```
int h(float x, float y, float z){
    if(y<x) z=-0.25+h(y*(3-x),0,z+x/2);
    else {
        int i=x;
        while(i){
            z=y*(z-2.5)/3;
            i=i*0.75;
        }
    }
    return z/5;
}
```

- MV u register
- addmul fcl.

// addmul fcl.

coll_dst	x	y	z	ret. addu	old_Fp	i	R2	R3	R4
#-5	#-4	#-3	#-2	#-1	#0	#1	#2	#3	#4

ENTER 1 + 3 //ver loc + m reg.

// if(y < x)

conv.i. f #3, 0

LESS. f #2, #-3, #-4
jF #2, L1 // else

conv.i. f #4, z
Div. f #4, #-4, #4
ADD. f #4, #2, #4

// z = ...

conv.i. f #2, 3
SUB. f #4, #2, #-4
MUL. f #2, #-3, #2
conv.i. f
ADD. f #-2, -0.25, #2

conv.i. f #2, 3
SUB. f #4, #2, #-4
MUL. f #2, #-3, #2

jmp L3
4 da

L1:

conv.f.i #1, #-4

//nohelp

L2:

IF #1, L3

SUB.f #2, #-2, 2.5

conv.i.f #3, 3

DIV.f #2, #2, #3

MUL.f #2, #-3, #2

conv.i.f #2, #1

MUL.f #2, #2, 0.15

conv.f.i #1, #2

jmp L2

L3:

conv.i.f #2, 5

DIV.f #2, #-2, #2

conv.f.i #2, #2

RET.i #2

③

```
// int f(int n, float x, float y){
int k=x*(y+2*(f(-x,0,y/(n+0.5))-1));
while (k<n){
k=x*(k-y);
if(k<0 x=-f(n,y,k)/2);
else if(k>0) x=x-1;
else return 0;
}
return k;
}
```

// initial ld.

call-addr	r	x	y	ret-addr	old-fp	k	r2	r3	r4
-5	-4	-3	-2	-1	0	1	2	3	4

ENTER 1+3

MUL. f #1, #-3, #2

// k = ...

// while

MUL. f #2, #-3, -1

L1:

conv. f.i #2, #2

LESS. i #2, #1, #4

JF #2, L2 // return

conv. i, f #3, 0

conv. i. f #2, #1

conv. i. f #4, #-4

SUB. f #2, #2, #-2

ADD. f #4, #4, 0.5

MUL. f #2, #-3, #2

DIV. f #4, #-2, #4

conv. f.i #1, #2

CALL. f #2, f, #2, #3, #4

// if

conv. i. f #3, 1

LESS. i #2, #1, 0

SUB. f #2, #2, #3

JF #2, L3 // else of

conv. i. f #3, 2

MUL. f #2, #3, #2

conv. i. f #2, #1

ADD. f #2, #-2, #2

CALL. f #2, f, #-4, #-2, #2

conv. i. f #3, 2

Div. f r_2, r_2, r_3

MUL. f $\#-3, -1, r_2$

JMP L1

// else of

L3:

GREATER. i $r_2, \#1, 0$

JF $r_2, L4$ // alt else

Conv. i. f $r_2, 1$

SUB. f $\#-3, \#-3, r_2$

JMP L1

// else

L4:

Conv. i. f $r_2, 0$

RET. f r_2

L2:

Conv. i. f $r_2, \#1$

RET. f r_2